



Teyzix Core Internship

Task Information

Task Title: Real-Time Ride Sharing System with Live Driver Tracking (Rider & Driver Apps)

Task ID: MAD-3

Domain: Mobile App Development

Difficulty Level: Intermediate

Assigned Date: 21st May 2026

Submission Deadline: 30th May 2026

Objective

Design and develop a dual mobile application system (Rider App + Driver App) that simulates a real-world ride-sharing platform with live GPS tracking, ride lifecycle management, and real-time communication.

The system must demonstrate real-time location streaming, ride state transitions, and dynamic fare calculation while maintaining smooth mobile performance and background execution reliability.

Problem Statement

Ride-sharing platforms depend heavily on real-time geolocation, low-latency communication, and reliable background tracking systems.

TEYZIX CORE requires a functional MVP of a ride-sharing ecosystem where:

- Riders can request rides and track drivers live
- Drivers can broadcast their location continuously
- Both parties can see ride status updates in real time

- Fare is dynamically calculated based on time and distance

Participants must build a production-style mobile system capable of handling real-time tracking, state management, and mobile background limitations.

Task Requirements

Participants must develop **two separate mobile applications**:

1. Driver Application

- Live GPS tracking
- Broadcast location every 3 seconds via WebSockets
- Accept / reject ride requests
- Ride status updates:
 - Available
 - On trip
 - Offline
- Background location tracking support
- Battery-efficient location updates
- Ride history view

2. Rider Application

- Map-based UI showing live driver location
- Ride booking system:
 - Request ride
 - Match driver
 - Track ride in real time
 - Complete ride flow
- Real-time driver movement updates
- Fare estimation display
- Ride history + receipts

3. Ride Lifecycle Management

System must support full state transitions:

- Ride Requested
- Driver Assigned
- Ride Accepted
- Ride In Progress

- Ride Completed
- Ride Cancelled

All transitions must be synchronized in real time across both apps.

Core Features

Real-Time Tracking

- WebSocket-based GPS streaming
- Live map updates using Google Maps SDK
- Smooth marker animation for driver movement

Fare Calculation

- Dynamic fare estimation based on:
 - Distance traveled
 - Time duration
 - Base fare logic
- Final fare calculation at ride completion

Notifications System

- Firebase Cloud Messaging (FCM)
- Push notifications for:
 - Ride request
 - Driver acceptance
 - Ride start
 - Ride completion

Background Processing

- Driver location updates even when app is minimized
- Efficient battery usage strategies
- Location throttling and optimization

Suggested Tools & Technologies

Participants may choose any suitable tools, frameworks, or architecture approach.

Mobile Development Frameworks

- Flutter

- React Native
- Kotlin (Android native)
- Swift (iOS native)

Real-Time Communication

- WebSockets
- Socket.io
- Firebase Realtime Database (alternative option)

Maps & Location Services

- Google Maps SDK
- Mapbox
- Geolocation APIs

Backend Options

- Node.js + Express
- Firebase Functions
- NestJS

Notifications

- Firebase Cloud Messaging (FCM)

Storage

- Firebase Firestore
- PostgreSQL
- MongoDB

Participants are free to use alternative tools as long as real-time tracking and ride lifecycle requirements are fully implemented.

Technical Constraints

- GPS updates must be sent every 3 seconds (driver app)
- Rider app must reflect live driver movement in real time
- Ride state must remain consistent across both apps
- WebSocket connection must handle reconnection automatically
- Background location tracking must function reliably
- Battery usage must be optimized for long-running tracking
- Apps must remain stable under poor network conditions

Expected Deliverables

Participants must submit:

- Complete GitHub repository (both apps)
- Full source code for Rider + Driver apps
- Backend source code (if applicable)
- System design document:
 - Real-time location pipeline
 - Ride matching flow
 - State transition architecture
- Demo video showing:
 - Ride request from rider
 - Driver acceptance
 - Live tracking in real time
 - Ride completion
- Battery optimization analysis report
- API documentation (if backend is used)
- Deployment/setup instructions

Submission Guidelines

Submission Format

`task-7_Red-ID_Name.zip`

The ZIP must include:

- Rider app source code
- Driver app source code
- Backend code (if any)
- Documentation
- Architecture diagrams
- Demo assets
- Setup instructions

All components must be structured professionally and ready for execution.

Evaluation Criteria

Evaluation will be based on:

- Real-time tracking accuracy and smoothness

- Reliability of WebSocket communication
- Ride lifecycle state management
- Mobile performance optimization
- UI/UX quality in both apps
- Background location tracking efficiency
- Battery optimization strategy
- Code structure and maintainability
- System design clarity
- Demo quality and completeness

Bonus Challenge

Implement advanced features such as:

- Driver-rider matchmaking algorithm (proximity-based)
- Surge pricing system
- Heatmap of driver density
- In-app chat between rider and driver
- Route optimization using Google Directions API
- Multi-stop rides
- Wallet & payment integration
- Anti-spoofing GPS detection
- Offline queueing of ride requests
- AI-based ETA prediction

Plagiarism Policy

Strict originality enforcement applies. Copying existing ride-sharing tutorials, GitHub clones, or template apps without meaningful architectural understanding and customization will lead to disqualification.

Participants must demonstrate understanding of real-time systems, mobile background execution, and location streaming challenges.

Support & Communication

For mobile development support:

[#mobile-dev-support](#)